

Plan-over-Graph: 面向可并行的 LLM 智能体调度

Shiqi Zhang^{1, #}, Xinbei Ma^{1, #}, Zouying Cao¹, Zhuosheng Zhang¹, Hai Zhao^{1, †}

¹Shanghai Jiao Tong University

zsqu259@sjtu.edu.cn, sjtumaxb@sjtu.edu.cn, zhaohai@cs.sjtu.edu.cn,

摘要

大语言模型 (LLM) 已在任务规划推理方面展现出突出能力, 但并行调度仍有许多未被充分研究的挑战。本文提出新范式 *plan-over-graph*: 模型先将现实生活中的文本任务分解为可执行子任务, 构建抽象任务图; 随后以该任务图为输入, 理解其结构并生成并行执行计划。为了增强模型在复杂、可扩展图上的规划能力, 我们设计了自动且可控的合成图生成流水线, 并提出两阶段训练方案。实验表明, *plan-over-graph* 显著改善了基于 API 的 LLM 和可训练开源 LLM 的任务表现。通过将复杂任务规范化为图, 本方法自然支持并行执行, 提高整体效率。代码和数据公开于 <https://github.com/zsq259/Plan-over-Graph>。

1 引言

大语言模型的显著进展 (OpenAI, 2023; Templeton et al., 2024; Yang et al., 2024a), 推动智能体在复杂交互任务中展现出较强能力 (Yao et al., 2022b,a; Xi et al., 2024; Ma et al., 2024; Yang et al., 2024b)。近期研究表明, 先生成计划再执行可以提高智能体表现; 这一框架称为 *plan-then-execute*, 即先规划再执行 (Zhao et al., 2024; Hao et al., 2023a; Liu et al., 2023; Zhang et al., 2023; Lin et al., 2024a)。规划整合全局知识, 使整体保持一致, 而非仅追求局部最优 (Qiao et al., 2024; Ruan et al., 2024)。它将复杂任务分解为以单步操作表示的子任务; 对于需要复杂工作流和精确动作接口的任务, 例如 UI 控制 (Hong et al., 2023; Wu et al., 2024; Zhang et al., 2024a) 和软件工程 (Yang et al., 2024c), 这一点尤其关键。

尽管已有进展令人鼓舞, 计划的并行性仍未得到充分研究。多步智能体框架通常默认采用阻塞流水线: 每一步都等待前面的步骤完成, 无论它是否依赖前面的结果 (Wu et al., 2024; Gou et al., 2024)。Chain-of-Thoughts (CoT) (Wei et al., 2023) 显著激发智能体推理能力,

同等贡献。

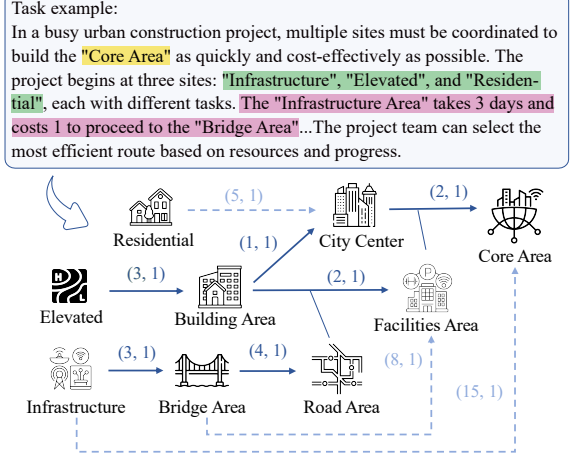


图 1: 本任务的示例: 从现实文本查询生成并行计划。计划表示为图。边表示可用规则, *Residential*, *Elevated* 和 *Infrastructure* 为初始源节点, *Core* 为目标节点。实线边表示以耗时为约束的最优计划。图内文字译释: *Task example* 为任务示例, 叙述为“在一个繁忙的城市建设项目中, 需要协调多个场地, 尽快且经济地建成核心区。项目从基础设施区、高架区和住宅区三个场地开始, 各有不同任务; 基础设施区到桥梁区需 3 天、成本为 1……项目团队可以根据资源与进度选择最高效路线”。*Residential* 为住宅区, *Elevated* 为高架区, *Infrastructure* 为基础设施区, *Bridge/Building/Road/City Center/Facilities/Core Area* 分别为桥梁区、建筑区、道路区、市中心区、设施区、核心区。括号数对表示耗时与成本。

使其能够分而治之地处理复杂任务。虽然推理结构已扩展为树和图 (Yao et al., 2023; Besta et al., 2024; Zhou et al., 2024), 子任务的动作仍顺序执行。然而, 如果这些子任务彼此独立, 就可以并行运行。近期研究虽然探索了异步执行的时间效率, 应用到现实场景时仍有差距。

本文深入研究复杂任务图下智能体规划的并行性, 如图 1 所示。我们提出 *plan-over-graph* 新范式: 智能体先探索规则、抽取图, 再在全局资源消耗约束下根据图结构制定计划。

为这一方法, 我们构建了包含并行子任务的复杂任务数据集。每个样本从一个连通有向无

环图出发，标注源节点、目标节点，以及可行解和最优解。随后，通过提示 LLM 生成文本描述，为这些图添加合适场景，最终得到自然语言形式的现实任务描述。

我们进一步通过可训练方案改进 *plan-over-graph*。图规模扩大时，智能体仍难以理解图 (Fatemi et al., 2023; Chen et al., 2024; Luo et al., 2024; Dai et al., 2024)，形成性能瓶颈。因此，我们在抽象图上采用两阶段训练策略。推理时，先提示智能体把文本查询抽取为图，再加载已训练适配器，在图上规划。

我们使用成功率、最优准确率、可行准确率和效率等综合指标衡量结果。实验在基于 API 的 LLM 和开源 LLM 上均取得显著改善。我们还分析图结构扩展的影响，展示并行执行如何提高时间效率，并识别任务规划中的常见错误。

本工作的贡献如下：

- 提出 *plan-over-graph*，支持子任务并行的规划，并构建复杂任务图数据集。
- 通过任务图训练增强 *plan-over-graph*，在多种 LLM 上获得显著改善。
- 分析本方法的规划效率，并展示其面对不同模型和图拓扑时的稳健性。

2 相关工作

本节介绍智能体规划和 LLM 图理解的背景。

2.1 基于 LLM 的智能体规划

自主智能体通过与环境交互解决复杂任务 (Yao et al., 2022a; Fan et al., 2022)。规划在执行前制定动作序列，利用任务和环境的全球知识，给出逻辑一致的轨迹 (Huang et al., 2024)。

规划分解复杂任务，并依据全局知识选择可行轨迹 (Valmeekam et al., 2023; Wang et al., 2023; Wu et al., 2024)。深度优先搜索、广度优先搜索和蒙特卡洛树搜索等策略用于探索最优计划 (Yao et al., 2023; Zhou et al., 2024; Qi et al., 2024; Zhao et al., 2024)。当环境反馈显著补充感知、更新任务知识时，规划会通过反思逐步改进轨迹 (Shinn et al., 2024)。世界模型或奖励模型整合全局知识，预测环境状态或估计奖励 (Hao et al., 2023b; Qiao et al., 2024)。

2.2 图上的 LLM

现实中的复杂挑战可以表示为图，因此近期研究探索了 LLM 在图上推理的能力。Graph-of-Thought (Besta et al., 2024; Ning et al., 2024) 首先提出将问题的思考过程转换为任意图，以支持子任务的生成、聚合和改进。Yao et al.

(2024) 也从上下文中抽取演绎三元组并构建图。知识图谱为知识密集型任务提供忠实性和推理透明度 (Luo et al., 2023; Sun et al., 2024; Wen et al., 2023)。Lin et al. (2024b) 将图与自然语言提示结合，用于现实任务中的异步计划推理：要求模型根据给定图推理，或先自行生成图再推理。

但研究表明，随着图规模和复杂度增加，图推理与理解能力会下降。实证研究观察到随图规模增大的“理解崩溃”现象 (Sui et al., 2023; Cao et al., 2024)。DARG (Zhang et al., 2024b) 评测 LLM 在图上的推理能力，也报告了图复杂度上升时的性能下降。

与已有工作不同，本文进一步给出更形式化、可扩展的规划任务图结构定义，刻画任务的内在复杂性与依赖。*planning-over-graph* 提供不依赖具体任务性质的通用框架。此外，我们通过这些图上训练模型，展示该方法能显著提高性能。

3 预备知识：问题定义

本节形式化定义任务图上的规划问题，并通过初步分析识别关键挑战。

3.1 规划的形式化定义

规划要求智能体将高层任务描述分解为可执行子任务，按照依赖调度执行，优化时间、成本或多目标指标，最终达到目标。形式化而言，给定任务描述，模型通过求解二元组 $\langle G, \Omega \rangle$ 生成计划 P ，其中 G 表示复杂任务， Ω 表示全局指标。¹

任一高层任务描述都可表示为有向无环图 (DAG)：

$$G = (T, E), \quad T = \{t_1, t_2, \dots, t_n\}, \quad (1)$$

其中，顶点受到边所形成的先后关系约束。例如， $t_i \prec t_j$ 表示 t_i 必须先于 t_j 。一个可行计划 P 是 G 的子图，且满足 G 的要求，即 $P \subset G$ 。

现实场景中还存在执行时间等约束指标。我们形式化定义全局函数 Ω ：最优计划在完成任务的同时最小化 Ω ，

$$P_{opt} = \arg \min_P \Omega. \quad (2)$$

评价指标 检查预测计划是否最优、或至少可行，并计算全局指标。具体而言，对于测试数据集 D 上的每个预测计划 $\hat{P}$ ，使用三类指标评估其有效性：(i) 最优率 (Optimal Rate)：最优计划的比例，

¹译注：下文的总完工时间模型只约束任务依赖，未加入执行资源容量或多个用户工作流之间的争用；这里的最优计划是在该抽象模型及给定规则耗时下定义的。

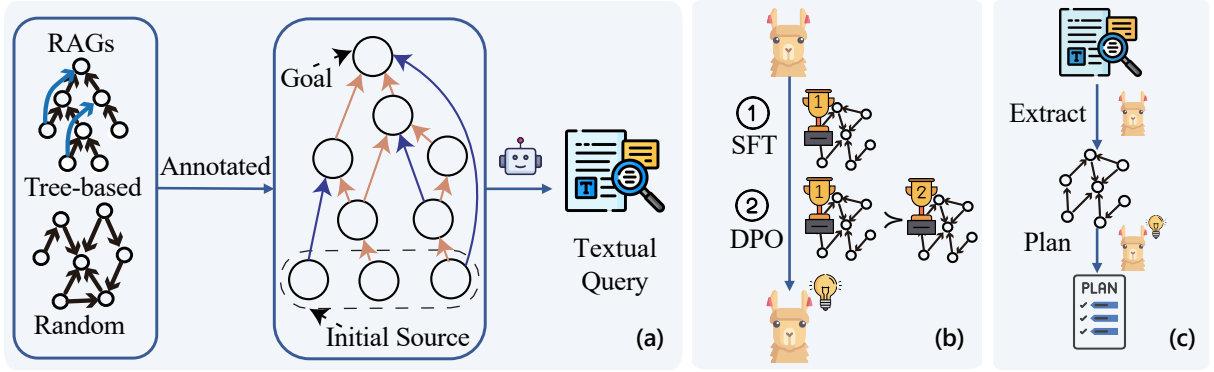


图 2: 框架概览。(a) 数据合成流水线; (b) 训练过程; (c) plan-over-graph 范式。图内文字译释: *Tree-based* 为基于树的图, *Random* 为随机图; *RAGs* 按原图标标注保留; *Annotated* 为标注; *Goal* 为目标; *Initial Source* 为初始源; *Textual Query* 为文本查询; *SFT* 为监督微调, *DPO* 为直接偏好优化; *Extract* 为抽取, *Plan* 为规划; 优于符号表示偏好最优解。

$$OR = |\hat{P} = P_{opt}|/|D|. \quad (3)$$

(ii) 成功率 (Success Rate): 成功达到目标的计划比例,

$$SR = |\hat{P} \in \{P\}|/|D|. \quad (4)$$

(iii) 所考察全局指标 Ω 的取值。

3.2 规划的挑战

现有探索仍有两项关键局限。(i) 图理解目前是 LLM 复杂任务规划的瓶颈。Lin et al. (2024b) 表明, 即使提供显式图表示, 模型处理复杂图拓扑的能力仍存在巨大差距, 说明结构推理还有未解决的挑战。(ii) 现有研究考虑的图规模十分有限。WorFBench (Qiao et al., 2025) 中, 大多数图只有 2–10 个步骤; AsyncHow (Lin et al., 2024b) 的图复杂度 $|V| + |E|$ 也大多介于 10 和 20 之间。

针对这些局限, 我们设计了先导实验: 构建具有 10、30、50 个节点的 100 个随机图, 要求 LLM 找出最短路径作为解。表 1 给出可行路径和最优路径的准确率, 结果随节点数增加急剧下降。尤其对于 50 节点图, 最优率只有 6%。

节点数	最优率 $\uparrow$	成功率 $\uparrow$
10	29.0	79.0
30	16.0	35.0
50	6.0	10.0

表 1: Llama-3.1-8B-Instruct 在随机图上的表现。

这些实证结果表明, 核心瓶颈是复杂约束下

对图拓扑的规划能力。因此, 我们优先提升模型对带约束复杂图的理解能力, 以改进规划任务。

4 方法: Plan-over-Graph

我们提出 *plan-over-graph* 范式。给定文本查询, 先提示 LLM 收集信息并构建任务图; 随后将任务图作为输入, 让模型在图上规划。接着, 我们着重增强 *plan* 阶段的图理解能力。

第 4.1 节和第 4.2 节提出数据构建方法, 自动获得大量可控图数据; 随后, 第 4.3 节用这些图数据设计训练流水线。最后, 在推理阶段结合图抽取与图规划, 形成 *plan-over-graph* 范式。整体框架见图 2。

4.1 抽象任务定义

首先, 我们在图结构上重新定义规划任务。不失一般性, 考虑 Ω 中的时间与成本约束, 要求计划最小化总完工时间 (makespan) 或总成本。

任务图。 我们定义支持图上并行执行的规则 $R = \{r_i\}$ 。对于每个节点,

$$r = (S, t, \tau, c). \quad (5)$$

规则表示: 在 S 得到满足后, 可以执行 t , 所需时间为 τ 、成本为 c 。

查询。 包含先验知识及最终目标的查询, 可以用初始节点集和目标节点表示为

$$q = (I, t_{\text{target}}), \quad (6)$$

计划。 整体计划 P 由若干子计划组成, 每个子计划表示为

$$p = (S, t, D), \quad p_j \in D_i \iff t_j \in S_i \quad (7)$$

其中, S 和 t 分别表示前序顶点及子任务, 由此确定一条规则 $r \in R$ 。 D 描述子计划之间的依赖关系。

评价准则。我们定义所考察的两个全局指标: 耗时与成本。相互独立的子任务可以并行执行。换言之, p_i 只需等待依赖结束即可开始。 p_i 执行完成的时间可以表示为:

$$\begin{aligned} \text{End_time}(p_i) &= \max_{p_j \in D_i} (\text{End_time}(p_j)) + \tau_{p_i}, \\ \Omega_{\text{time}}(P) &= \max_{p \in P} (\text{End_time}(p)). \end{aligned} \quad (8)$$

全局成本定义为全部子任务成本之和:

$$\Omega_{\text{cost}}(P) = \sum_{p \in P} c_p. \quad (9)$$

我们主要考虑耗时。因此, P_{opt} 是使下式最小的计划:

$$\Omega(P) = \Omega_{\text{time}}(P) + \epsilon \cdot \Omega_{\text{cost}}(P) \quad (10)$$

其中 $\epsilon \ll 1$ 。

按上述图定义, 通过时间效率 (TE) 与成本效率 (CE) 衡量 Ω :²

$$\begin{aligned} \text{TE} &= \frac{1}{n} \sum_{i=1}^n \frac{\Omega_{\text{time}}(P_i)}{\Omega_{\text{time}}(P_{\text{opt}})}, \\ \text{CE} &= \frac{1}{n} \sum_{i=1}^n \frac{\Omega_{\text{cost}}(P_i)}{\Omega_{\text{cost}}(P_{\text{opt}})}. \end{aligned} \quad (11)$$

对于失败计划, 时间比和成本比均设为 4, 作为惩罚。

4.2 数据模拟

依照上述定义, 我们设计自动、可控、可扩展的合成数据生成流水线, 包含以下步骤: ○ 生成连通 DAG。我们使用两种不同图结构: (i) 随机 DAG。

(ii) 为更好地符合现实中的层次结构, 避免随机图中出现大量捷径, 我们构建基于树的结构。先生成深度不超过 4 的树, 再加入少量祖先边和跨分支边, 增加依赖、丰富图结构; 所有边均指向根节点。第 6.1 节分析两种表示的结构权衡。我们确保图连通, 即初始顶点到目标之间至少存在一条路径。

○ 定义规则。对于 DAG 中每个非头部子任务 t , 将其前驱顶点尽可能均匀地随机分组。每个前驱组与 t 形成一条规则的源和目标。为每条规则分配从 1 到 50 的均匀分布采样所得的

²译注: TE、CE 的公式是相对于最优计划的平均时间与成本比, 数值越小越好; 它们虽被命名为 efficiency, 含义并非通常越大越好的吞吐效率。

随机耗时。所有规则的成本固定为 1, 以简化成本结构, 并聚焦时间优化。

○ 定义初始节点和目标节点。DAG 中所有头部顶点均设为整个任务的初始源顶点; 从尾部顶点中随机选择一个目标, 确保任务目标明确。

○ 标注最优解与可行解。应用动态规划算法计算图中各个解的标签, 以最优解作为标准答案。

○ 构建查询描述。根据上述图生成文本查询。我们提示 LLM 将图转换为现实场景描述; 为保证任务与原图一致, 让 LLM 自我纠正, 核对查询描述是否与原图匹配。

4.3 训练方案

本节优化模型在抽象图上的能力, 增强其并行规划能力。训练分为监督微调与直接偏好优化两个阶段。**监督微调。**我们在抽象任务数据集上微调 LLM, 使模型能够用问题空间的图表示求解规划任务。采用 Low-Rank Adaptation (LoRA) (Hu et al., 2022), 通过学习小型适配器, 高效调整大型预训练模型。监督微调有两种设置: (i) 仅使用最优数据实例微调; (ii) 为了让模型同时学习最优解和可行解, 选取次优解, 与最优解混合作为训练数据。

直接偏好优化。微调后, 使用直接偏好优化 (DPO) (Rafailov et al., 2023) 区分最优解与可行解。每个样本中, 次优解作为不偏好的输出, 最优解作为偏好的输出。该步骤进一步增强模型优先选择最优解的能力。

训练完成后, 推理阶段结合抽取与规划。给定含目标描述的查询, 先从描述中抽取任务图, 再加载已训练适配器, 在图上生成计划。

5 实验

5.1 数据集

任务图。节点数为 n 的连通图, 其边数理论范围为 $n-1$ 到 $n(n-1)/2$ 。当 n 较大时, 大量边导致输入过长; 基于树的结构也不宜具有太多边。因此, 我们采用两种实用策略: (i) 线性扩展, 随机图的边数为 $\in [2n, 3n]$, 基于树的图为 $\in [n, 1.5n]$; (ii) 在完整边数范围上均匀分布, 该策略仅用于随机图。

表 2 给出合成数据统计。训练集含 12,000 个实例, 平均分布在三个节点规模 (10、30、50) 以及随机 DAG、基于树的 DAG 两种结构上。10/30 节点图采用均匀边数配置, 50 节点图限制为线性扩展。每个节点规模与图结构生成 1000 个输入, 每个输入对应一个最优解和一个选定可行解。测试集有两部分: (i) 基线测试, 节点数为 10、20、30、40、50, 边数线

集合	节点数	图结构	边数配置	样本数
训练	10	随机图 基于树	均匀 线性	2000 2000
	30	随机图 基于树	均匀 线性	2000 2000
	50	随机图 基于树	线性 线性	2000 2000
测试	10	随机图 基于树 随机图	线性 线性 均匀	100 100 1000
	20	随机图 基于树	线性 线性	100 100
	30	随机图 基于树 随机图	线性 线性 均匀	100 100 1000
	40	随机图 基于树	线性 线性	100 100
	50	随机图 基于树	线性 线性	100 100

表 2: 训练和测试数据集的详细统计。

性扩展；每个节点规模与图结构各有 100 个实例。(ii) 针对 10/30 节点随机图的边数变化测试，边数均匀分布，用于评估模型在边数变化时的图理解能力。由于边数变化范围较大，每种规模生成 1,000 个实例。

文本查询。为了系统评估模型在现实任务场景中的并行规划能力，并验证 *plan-over-graph* 范式，我们用 DeepSeek-R1 (DeepSeek-AI et al., 2025) 构建评测数据集。该数据集合成了来自若干现实问题领域的 200 个任务，每个任务说明均由图转换为可执行工作流描述。查询数据统计见附录 B。

5.2 基线

我们与多种基线模型比较，包括基于 API 的 GPT-4o (Hurst et al., 2024)、Claude 3.5 Sonnet (Anthropic, 2024)，以及开源的 Llama-3.1-8B-Instruct (Dubey et al., 2024) 和 Qwen2.5-7B-Instruct (Yang et al., 2024a)。这些模型表现较强且被广泛使用。评价指标详见第 4.1 节，实验设置详见附录 C。

5.3 主要结果

表 3 给出实验结果，回答以下三个关键问题。

问题 1: 训练方法能否教会 LLM 在图上规划？哪种方法表现最好？各模型的总体结果见表 3 上半部分。总体而言，**训练大幅提高规划表现，两阶段训练取得更好的结果**。未训练时，Claude 成功率较高 (90.0%)，但最优率较低 (39.2%)。GPT-4o 与 Llama 的成功率相

近 (51.3% 与 52.3%)，但 GPT-4o 的最优率为 14.1%，高于 Llama 的 1.8%。Qwen 的成功率和最优率都较弱，分别为 13.2% 和 0.5%。

仅用最优解训练即可显著改善图理解和规划能力，使 Llama 成功率达到 75.7%、最优率达到 61.4%。混入可行解后，进一步提高至 86.1% 和 67.5%。最佳最优率为 71.6%，由“混合数据 SFT 加 DPO”的两阶段训练取得，同时保持 83.6% 的较高成功率。这是因为 DPO 使模型更倾向于选择最优解。我们也在表现弱于 Llama 的 Qwen 上观察到一致的改善趋势。

问题 2: 图规模扩大带来什么现象？如图 3 上半部分所示，节点数增加、图结构扩大时，所有模型整体表现都下降。各模型的时间比和成本比对节点数具有相似敏感性；其中 Llama 的成本比增加尤其明显，说明它在更大的图上倾向于选择更多子任务。GPT-4o 与 Llama 的成功率下降较快；节点数达到 30 时，GPT-4o 甚至低于 Llama。不过，Claude 和已训练模型仍保持较强能力，下降较缓。Claude 的最优率仍下降 10–20 个点，说明其理解更大图仍有不足。在全部节点规模上，已训练 Llama 的最优率显著优于其他模型。图 3 下半部分给出 10/30 节点、各 1000 个实例、完整边数范围下 Claude 与已训练 Llama 的结果。10 节点时，由于规模较小，两种模型各项指标对边数变化都较稳健。30 节点时，随着边数增加，寻找最优解的难度明显上升，因此两种模型的平均时间比都增大；平均成本比的增幅较小，因为图越密，可行解越多，模型可以选择较少子任务完成任务，但耗时未必最优。

问题 3: *plan-over-graph* 能否改善文本查询上的规划表现？表 3 下半部分给出现实查询结果，**显示 *plan-over-graph* 始终改善不同模型的表现**。不先抽取图而直接规划时，Claude 成功率为 89.5%，但最优率只有 14.5%，时间比较高，为 1.904；Llama 成功率为 19%，没有得到最优计划，时间比为 3.433。使用 *plan-over-graph* 后，Claude 最优率提高到 41.5%，Llama 成功率提高到 38%，二者时间比均下降。经规划训练的模型表现显著改善：最优率达到 72.5%，甚至超过 Claude，成功率为 83%。这些结果表明，*plan-over-graph* 改善模型表现，而对规划能力进行训练会进一步显著增强表现。

6 分析

本节讨论数据集及实验的详细结果。

6.1 图特征

本节讨论：(i) 图结构设计的考量；(ii) 图节点数和边数的变化对模型规划能力的影响。**图结构。**基于树的结构旨在更好地反映现实并行

总体结果					
模型	最优率 ↑	成功率 ↑	可行率	平均时间比 ↓	平均成本比 ↓
Claude 3.5 Sonnet	39.2	90.0	50.8	1.545	1.589
GPT-4o	14.1	51.3	37.2	2.657	2.889
Llama-3.1-8B-Instruct	1.8	52.3	50.5	2.616	4.512
Llama-3.1-8B-Instruct _{opt} SFT	61.4+59.6	75.7+23.4	14.3	1.746-0.870	1.769-2.743
Llama-3.1-8B-Instruct _{opt+feas} SFT	67.5+65.7	86.1+33.8	18.6	1.507-1.109	1.498 -3.014
Llama-3.1-8B-Instruct _{opt+feas} SFT + opt DPO	71.6 +69.8	83.6+31.3	12.0	1.502 -1.114	1.520-2.992
Qwen2.5-7B-Instruct	0.5	13.2	12.7	3.612	4.072
Qwen2.5-7B-Instruct _{opt+feas} SFT + opt DPO	27.0+26.5	75.8+62.6	48.8	2.191-1.421	2.029-2.043

文本查询结果					
方法	最优率 ↑	成功率 ↑	可行率	平均时间比 ↓	平均成本比 ↓
Claude 规划	14.5	89.5	75.0	1.904	2.302
Claude 抽取 + 规划	41.5+27.0	93.5 +4.0	52.0	1.514 -0.390	1.689-0.613
Llama 规划	0.0	19.0	19.0	3.433	4.103
Llama 抽取 + 规划	3.5+3.5	38.0+19.0	34.5	2.952-0.481	3.553-0.550
Llama 抽取 + 已训练 Llama 规划	72.5 +72.5	83.0+64.0	17.0	1.540-1.893	1.526 -2.577

表 3: 实验结果：上半部分为全部基线测试集的结果，下半部分为现实任务的结果。表内缩写：opt 为最优解，feas 为可行解，SFT 为监督微调，DPO 为直接偏好优化。译注：最后一行的成功率 83.0、最优率 72.5、可行率 17.0 不满足前文按“最优加可行”划分成功计划的关系；本译文保留三项原数值。

场景，通过更强的层次组织生成更合理的具体场景。为了清楚区分并行与非并行执行，我们采用以下策略：(i) 控制树深度以管理分支数量；(ii) 节点分组时，每组节点数不超过全部前驱节点的三分之二。此外，我们也采用未预设结构的随机图来验证模型的稳健性。

节点数和边数的影响。我们计算四项归一化指标随节点数与边数变化的相关系数和斜率的绝对值。整体上，边数的影响小于节点数。对于节点数，几乎所有模型指标的相关系数都较强，介于 0.8 和 1.0 之间。对于 10 节点图的边数变化，两种模型的相关系数较低，小于 0.5，说明小图上的表现对边数变化较稳健；但对于 30 节点图，Claude 的四项指标相关系数都高于已训练 Llama，超过 0.7，表现出较低稳定性。细节见附录 F。

6.2 时间效率

这些任务本身支持子任务并行，但多数已有方法在规划时不考虑并行性，造成不必要的等待。我们计算计划的并行执行时间与顺序执行时间（即所有子任务耗时之和）的比值。表 4 给出最优标注，以及已训练 Llama 和 Qwen 的结果。结果表明，相对于阻塞的顺序执行，规划并行方案能够显著缩短时间；图规模越大，这种效率越明显。Llama 和 Qwen 的计划成本比较高，说明有许多冗余子任务。顺序执行这些计划会进一步放大低效，因而并行时间与顺序时间的比值更低。

6.3 错误案例研究

任务图。抽象图上的错误分为两类：(i) 无效子任务，即计划中包含没有对应转换规则的子

节点数	最优解		已训练 Llama		已训练 Qwen	
	R	T	R	T	R	T
10	0.88	0.92	0.88	0.92	0.88	0.93
20	0.76	0.74	0.77	0.80	0.79	0.79
30	0.74	0.75	0.75	0.75	0.73	0.68
40	0.70	0.68	0.71	0.68	0.68	0.61
50	0.68	0.62	0.73	0.61	0.70	0.56

表 4: 各模型在测试实例上提供的计划，其并行执行时间与顺序执行时间的比值。R 为随机图，T 为基于树的图。

任务；(ii) 源不可用，即执行中未获得某个子任务所需的源。后一类意味着规划时没有考虑源的可用性，或错误处理依赖。表 5 给出两类错误的比例。训练后，源依赖问题基本得到解决；当前性能瓶颈主要是产生无效子任务的幻觉。

模型	无效子任务	不可用源
Claude 3.5 Sonnet	0.4	9.6
GPT-4o	4.7	44.0
Llama-3.1-8B-Instruct	17.6	30.1
Llama-3.1-8B-Instruct-Trained	11.6	4.8
Qwen2.5-7B-Instruct	60.7	26.1
Qwen2.5-7B-Instruct-Trained	19.9	4.3

表 5: 两类错误原因在各模型全部测试实例中所占的比例。

文本查询。如果未能抽取子任务的必要规则，模型整体表现会受到影响。不过，即使抽取有误，只要后续规划没有使用错误规则，模型仍可能正确完成任务。Llama 的结果表明，抽取步骤显著提高基线成功率；已训练模型的成功率虽略低于 Claude，最优率却更高。进一步检查发现，只有 15% 的抽取结果与原图完全一

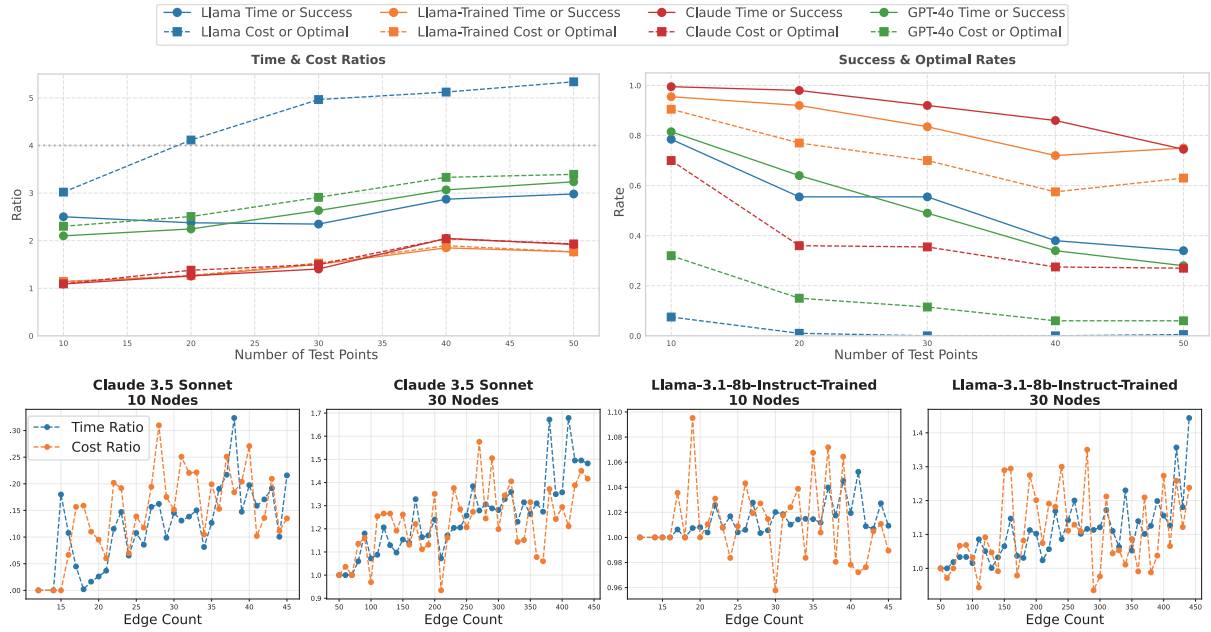


图 3: 上半部分展示不同节点数下的模型表现: 左图为时间比、成本比随节点数的变化, 右图 of 成功率和最优率。下半部分展示不同边数下, Claude 与已训练 Llama 的平均时间比和成本比。图内文字译释: *Time & Cost Ratios* 为时间比与成本比; *Success & Optimal Rates* 为成功率与最优率; 横轴 *Number of Test Points* 表示节点数, 纵轴 *Ratio/Rate* 分别为比值/比例。上方图例中的 *Time or Success* 表示时间比或成功率, *Cost or Optimal* 表示成本比或最优率; *Llama-Trained* 为已训练 Llama。下方为各模型的 10/30 节点结果, 横轴 *Edge Count* 为边数, *Time Ratio/Cost Ratio* 分别为时间比和成本比。

致; 但不匹配实例的平均相似度为 82%, 表明影响较小。这支持我们重点提升模型在抽象图上的规划能力。

7 结论

我们提出 plan-over-graph, 为基于 LLM 的智能体规划增强并行性。该方法将任务依赖抽取为结构化图, 再通过图感知推理优化并行规划。我们构建以 DAG 标注的合成数据集, 并提出两阶段训练方案; 实验表明性能显著改善。分析进一步揭示, 图结构对模型表现与并行性带来的时间缩短存在反向影响。本工作为并行智能体系统建立了框架, 连接抽象图与现实应用。

局限性

本工作存在以下局限: (i) 虽然我们认为并验证了图上规划能力比抽取更重要, 开源模型在抽取方面仍有不足。(ii) 现实中的模型规划可以是与环境交互的动态过程, 模型能够通过感知改进先前计划。未来将着重研究这两个方向。

参考文献

Anthropic. 2024. Claude 3.5 sonnet. <https://www.anthropic.com/news/claude-3-5-sonnet>.

Maciej Besta, Nils Blach, Ales Kubicek, Robert Gerstenberger, Lukas Gianinazzi, Joanna Gajda, Tomasz Lehmann, Michał Podstawski, Hubert Niewiadomski, Piotr Nyczyk, and Torsten Hoefler. 2024. *Graph of Thoughts: Solving Elaborate Problems with Large Language Models*. *Proceedings of the AAAI Conference on Artificial Intelligence*, 38(16):17682–17690.

Yukun Cao, Shuo Han, Zengyi Gao, Zezhong Ding, Xike Xie, and S. Kevin Zhou. 2024. *Graphinsight: Unlocking insights in large language models for graph structure understanding*. *ArXiv*, abs/2409.03258.

Runjin Chen, Tong Zhao, Ajay Kumar Jaiswal, Neil Shah, and Zhangyang Wang. 2024. *Llaga: Large language and graph assistant*. *ArXiv*, abs/2402.08170.

Xinnan Dai, Haohao Qu, Yifen Shen, Bohang Zhang, Qihao Wen, Wenqi Fan, Dongsheng Li, Jiliang Tang, and Caihua Shan. 2024. *How do large language models understand graph patterns? a benchmark for graph pattern comprehension*. *ArXiv*, abs/2410.05298.

DeepSeek-AI, Daya Guo, Dejian Yang, Haowei Zhang, Junxiao Song, Ruoyu Zhang, Runxin Xu, Qihao Zhu, Shirong Ma, Peiyi Wang, Xiao Bi, Xiaokang Zhang, Xingkai Yu, Yu Wu, Z. F. Wu, Zhibin Gou, Zhihong Shao, Zhuoshu Li, Ziyi Gao, Aixin Liu, Bing Xue, Bingx-

- uan Wang, Bochao Wu, Bei Feng, Chengda Lu, Chenggang Zhao, Chengqi Deng, Chenyu Zhang, Chong Ruan, Damai Dai, Deli Chen, Dongjie Ji, Erhang Li, Fangyun Lin, Fucong Dai, Fuli Luo, Guangbo Hao, Guanting Chen, Guowei Li, H. Zhang, Han Bao, Hanwei Xu, Haocheng Wang, Honghui Ding, Huajian Xin, Huazuo Gao, Hui Qu, Hui Li, Jianzhong Guo, Jiashi Li, Jiawei Wang, Jingchang Chen, Jingyang Yuan, Junjie Qiu, Junlong Li, J. L. Cai, Jiaqi Ni, Jian Liang, Jin Chen, Kai Dong, Kai Hu, Kaige Gao, Kang Guan, Kexin Huang, Kuai Yu, Lean Wang, Lecong Zhang, Liang Zhao, Litong Wang, Liyue Zhang, Lei Xu, Leyi Xia, Mingchuan Zhang, Minghua Zhang, Minghui Tang, Meng Li, Miao-jun Wang, Mingming Li, Ning Tian, Panpan Huang, Peng Zhang, Qiancheng Wang, Qinyu Chen, Qiushi Du, Ruiqi Ge, Ruisong Zhang, Ruizhe Pan, Runji Wang, R. J. Chen, R. L. Jin, Ruyi Chen, Shanghao Lu, Shangyan Zhou, Shanhua Chen, Shengfeng Ye, Shiyu Wang, Shuiping Yu, Shunfeng Zhou, Shuting Pan, S. S. Li, Shuang Zhou, Shaoqing Wu, Shengfeng Ye, Tao Yun, Tian Pei, Tianyu Sun, T. Wang, Wangding Zeng, Wanbiao Zhao, Wen Liu, Wenfeng Liang, Wenjun Gao, Wenqin Yu, Wentao Zhang, W. L. Xiao, Wei An, Xiaodong Liu, Xiaohan Wang, Xiaokang Chen, Xiaotao Nie, Xin Cheng, Xin Liu, Xin Xie, Xingchao Liu, Xinyu Yang, Xinyuan Li, Xuecheng Su, Xuheng Lin, X. Q. Li, Xiangyue Jin, Xiaojin Shen, Xiaosha Chen, Xiaowen Sun, Xiaoxiang Wang, Xinnan Song, Xinyi Zhou, Xianzu Wang, Xinxia Shan, Y. K. Li, Y. Q. Wang, Y. X. Wei, Yang Zhang, Yanhong Xu, Yao Li, Yao Zhao, Yaofeng Sun, Yaohui Wang, Yi Yu, Yichao Zhang, Yifan Shi, Yiliang Xiong, Ying He, Yishi Piao, Yisong Wang, Yixuan Tan, Yiyang Ma, Yiyuan Liu, Yongqiang Guo, Yuan Ou, Yuduan Wang, Yue Gong, Yuheng Zou, Yujia He, Yunfan Xiong, Yuxiang Luo, Yuxiang You, Yuxuan Liu, Yuyang Zhou, Y. X. Zhu, Yanhong Xu, Yanping Huang, Yaohui Li, Yi Zheng, Yuchen Zhu, Yunxian Ma, Ying Tang, Yukun Zha, Yuting Yan, Z. Z. Ren, Zehui Ren, Zhangli Sha, Zhe Fu, Zhean Xu, Zhenda Xie, Zhengyan Zhang, Zhewen Hao, Zhicheng Ma, Zhigang Yan, Zhiyu Wu, Zihui Gu, Zijia Zhu, Zijun Liu, Zilin Li, Ziwei Xie, Ziyang Song, Zizheng Pan, Zhen Huang, Zhipeng Xu, Zhongyu Zhang, and Zhen Zhang. 2025. *Deepseek-r1: Incentivizing reasoning capability in llms via reinforcement learning*. Preprint, arXiv:2501.12948.
- Abhimanyu Dubey, Abhinav Jauhri, Abhinav Pandey, Abhishek Kadian, Ahmad Al-Dahle, Aiesha Letman, Akhil Mathur, Alan Schelten, Amy Yang, Angela Fan, et al. 2024. The llama 3 herd of models. *arXiv preprint arXiv:2407.21783*.
- Linxi Fan, Guanzhi Wang, Yunfan Jiang, Ajay Mandlekar, Yuncong Yang, Haoyi Zhu, Andrew Tang, De-An Huang, Yuke Zhu, and Aniruma Anandkumar. 2022. Minedojo: Building open-ended embodied agents with internet-scale knowledge. *Advances in Neural Information Processing Systems*, 35:18343–18362.
- Bahare Fatemi, Jonathan J. Halcrow, and Bryan Perozzi. 2023. *Talk like a graph: Encoding graphs for large language models*. *ArXiv*, abs/2310.04560.
- Zhibin Gou, Zhihong Shao, Yeyun Gong, yelong shen, Yujiu Yang, Nan Duan, and Weizhu Chen. 2024. *CRITIC: Large language models can self-correct with tool-interactive critiquing*. In *The Twelfth International Conference on Learning Representations*.
- Shibo Hao, Yi Gu, Haodi Ma, Joshua Hong, Zhen Wang, Daisy Wang, and Zhiting Hu. 2023a. Reasoning with language model is planning with world model. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pages 8154–8173.
- Shibo Hao, Yi Gu, Haodi Ma, Joshua Hong, Zhen Wang, Daisy Wang, and Zhiting Hu. 2023b. *Reasoning with language model is planning with world model*. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pages 8154–8173, Singapore. Association for Computational Linguistics.
- Wenyi Hong, Weihang Wang, Qingsong Lv, Jiazhen Xu, Wenmeng Yu, Junhui Ji, Yan Wang, Zihan Wang, Yuxiao Dong, Ming Ding, et al. 2023. *Cogagent: A visual language model for gui agents*. *ArXiv preprint*, abs/2312.08914.
- Edward J Hu, yelong shen, Phillip Wallis, Zeyuan Allen-Zhu, Yanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. 2022. *LoRA: Low-rank adaptation of large language models*. In *International Conference on Learning Representations*.
- Xu Huang, Weiwen Liu, Xiaolong Chen, Xingmei Wang, Hao Wang, Defu Lian, Yasheng Wang, Ruiming Tang, and Enhong Chen. 2024. Understanding the planning of llm agents: A survey. *arXiv preprint arXiv:2402.02716*.
- Aaron Hurst, Adam Lerer, Adam P Goucher, Adam Perelman, Aditya Ramesh, Aidan Clark, AJ Ostrow, Akila Welihinda, Alan Hayes, Alec Radford, et al. 2024. Gpt-4o system card. *arXiv preprint arXiv:2410.21276*.
- Bill Yuchen Lin, Yicheng Fu, Karina Yang, Faeze Brahman, Shiyu Huang, Chandra Bhagavatula, Prithviraj Ammanabrolu, Yejin Choi, and Xiang Ren. 2024a. Swiftsage: A generative agent with fast and slow thinking for complex interactive tasks. *Advances in Neural Information Processing Systems*, 36.
- Fangru Lin, Emanuele La Malfa, Valentin Hofmann, Elle Michelle Yang, Anthony G. Cohn,

- and Janet B. Pierrehumbert. 2024b. [Graph-enhanced large language models in asynchronous plan reasoning](#). In *Forty-first International Conference on Machine Learning*.
- Bo Liu, Yuqian Jiang, Xiaohan Zhang, Qiang Liu, Shiqi Zhang, Joydeep Biswas, and Peter Stone. 2023. Llm+ p: Empowering large language models with optimal planning proficiency. *arXiv preprint arXiv:2304.11477*.
- Linhao Luo, Yuan-Fang Li, Gholamreza Haffari, and Shirui Pan. 2023. [Reasoning on graphs: Faithful and interpretable large language model reasoning](#). *ArXiv*, abs/2310.01061.
- Zihan Luo, Xiran Song, Hong Huang, Jianxun Lian, Chenhao Zhang, Jinqi Jiang, Xing Xie, and Hai Jin. 2024. [Graphinstruct: Empowering large language models with graph understanding and reasoning capability](#). *ArXiv*, abs/2403.04483.
- Xinbei Ma, Zhuosheng Zhang, and Hai Zhao. 2024. [CoCo-agent: A comprehensive cognitive MLLM agent for smartphone GUI automation](#). In *Findings of the Association for Computational Linguistics: ACL 2024*, pages 9097–9110, Bangkok, Thailand. Association for Computational Linguistics.
- Xinyu Ning, Yutong Zhao, Yitong Liu, and Hongwen Yang. 2024. [Dgot: Dynamic graph of thoughts for scientific abstract generation](#). *ArXiv*, abs/2403.17491.
- OpenAI. 2023. [Gpt-4 technical report](#). *ArXiv preprint*, abs/2303.08774.
- Zhenting Qi, Mingyuan Ma, Jiahang Xu, Li Lyna Zhang, Fan Yang, and Mao Yang. 2024. [Mutual reasoning makes smaller llms stronger problem-solvers](#). *Preprint*, arXiv:2408.06195.
- Shuofei Qiao, Runnan Fang, Zhisong Qiu, Xiaobin Wang, Ningyu Zhang, Yong Jiang, Pengjun Xie, Fei Huang, and Huajun Chen. 2025. [Benchmarking agentic workflow generation](#). In *The Thirteenth International Conference on Learning Representations*.
- Shuofei Qiao, Runnan Fang, Ningyu Zhang, Yuqi Zhu, Xiang Chen, Shumin Deng, Yong Jiang, Pengjun Xie, Fei Huang, and Huajun Chen. 2024. [Agent planning with world knowledge model](#). In *The Thirty-eighth Annual Conference on Neural Information Processing Systems*.
- Rafael Rafailov, Archit Sharma, Eric Mitchell, Christopher D Manning, Stefano Ermon, and Chelsea Finn. 2023. [Direct preference optimization: Your language model is secretly a reward model](#). In *Thirty-seventh Conference on Neural Information Processing Systems*.
- Yangjun Ruan, Honghua Dong, Andrew Wang, Silviu Pitis, Yongchao Zhou, Jimmy Ba, Yann Dubois, Chris J. Maddison, and Tatsunori Hashimoto. 2024. [Identifying the risks of LM agents with an LM-emulated sandbox](#). In *The Twelfth International Conference on Learning Representations*.
- Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. 2024. Reflexion: Language agents with verbal reinforcement learning. *Advances in Neural Information Processing Systems*, 36.
- Yuan Sui, Mengyu Zhou, Mingjie Zhou, Shi Han, and Dongmei Zhang. 2023. [Table meets llm: Can large language models understand structured table data? a benchmark and empirical study](#). *Proceedings of the 17th ACM International Conference on Web Search and Data Mining*.
- Jiashuo Sun, Chengjin Xu, Lumingyuan Tang, Saizhuo Wang, Chen Lin, Yeyun Gong, Lionel Ni, Heung-Yeung Shum, and Jian Guo. 2024. [Think-on-graph: Deep and responsible reasoning of large language model on knowledge graph](#). In *The Twelfth International Conference on Learning Representations*.
- Adly Templeton, Tom Conerly, Jonathan Marcus, Jack Lindsey, Trenton Bricken, Brian Chen, Adam Pearce, Craig Citro, Emmanuel Ameisen, Andy Jones, Hoagy Cunningham, Nicholas L Turner, Callum McDougall, Monte MacDiarmid, C. Daniel Freeman, Theodore R. Sumers, Edward Rees, Joshua Batson, Adam Jermy, Shan Carter, Chris Olah, and Tom Henighan. 2024. [Scaling monosemanticity: Extracting interpretable features from claude 3 sonnet](#). *Transformer Circuits Thread*.
- Karthik Valmeekam, Matthew Marquez, Sarath Sreedharan, and Subbarao Kambhampati. 2023. [On the planning abilities of large language models - a critical investigation](#). In *Thirty-seventh Conference on Neural Information Processing Systems*.
- Lei Wang, Wanyu Xu, Yihuai Lan, Zhiqiang Hu, Yunshi Lan, Roy Ka-Wei Lee, and Ee-Peng Lim. 2023. Plan-and-solve prompting: Improving zero-shot chain-of-thought reasoning by large language models. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 2609–2634.
- Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Fei Xia, Ed Chi, Quoc V Le, Denny Zhou, et al. 2023. Chain-of-thought prompting elicits reasoning in large language models. *Advances in Neural Information Processing Systems*, 35:24824–24837.

- Yilin Wen, Zifeng Wang, and Jimeng Sun. 2023. [Mindmap: Knowledge graph prompting sparks graph of thoughts in large language models](#). In *Annual Meeting of the Association for Computational Linguistics*.
- Zhiyong Wu, Chengcheng Han, Zichen Ding, Zhenmin Weng, Zhoumianze Liu, Shunyu Yao, Tao Yu, and Lingpeng Kong. 2024. [OS-copilot: Towards generalist computer agents with self-improvement](#). In *ICLR 2024 Workshop on Large Language Model (LLM) Agents*.
- Zhiheng Xi, Yiwen Ding, Wenxiang Chen, Boyang Hong, Honglin Guo, Junzhe Wang, Dingwen Yang, Chenyang Liao, Xin Guo, Wei He, Songyang Gao, Lu Chen, Rui Zheng, Yicheng Zou, Tao Gui, Qi Zhang, Xipeng Qiu, Xuanjing Huang, Zuxuan Wu, and Yu-Gang Jiang. 2024. [Agentgym: Evolving large language model-based agents across diverse environments](#). *Preprint*, arXiv:2406.04151.
- An Yang, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chengyuan Li, Dayiheng Liu, Fei Huang, Haoran Wei, Huan Lin, Jian Yang, Jianhong Tu, Jianwei Zhang, Jianxin Yang, Jiayi Yang, Jingren Zhou, Junyang Lin, Kai Dang, Keming Lu, Keqin Bao, Kexin Yang, Le Yu, Mei Li, Mingfeng Xue, Pei Zhang, Qin Zhu, Rui Men, Runji Lin, Tianhao Li, Tingyu Xia, Xingzhang Ren, Xuancheng Ren, Yang Fan, Yang Su, Yichang Zhang, Yu Wan, Yuqiong Liu, Zeyu Cui, Zhenru Zhang, and Zihan Qiu. 2024a. Qwen2.5 technical report. *arXiv preprint arXiv:2412.15115*.
- John Yang, Carlos E Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. 2024b. [Swe-agent: Agent-computer interfaces enable automated software engineering](#). *arXiv preprint arXiv:2405.15793*.
- John Yang, Carlos E Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik R Narasimhan, and Ofir Press. 2024c. [SWE-agent: Agent-computer interfaces enable automated software engineering](#). In *The Thirty-eighth Annual Conference on Neural Information Processing Systems*.
- Shunyu Yao, Howard Chen, John Yang, and Karthik Narasimhan. 2022a. [Webshop: Towards scalable real-world web interaction with grounded language agents](#). *Advances in Neural Information Processing Systems*, 35:20744–20757.
- Shunyu Yao, Dian Yu, Jeffrey Zhao, Izhak Shafran, Thomas L. Griffiths, Yuan Cao, and Karthik R Narasimhan. 2023. [Tree of thoughts: Deliberate problem solving with large language models](#). In *Thirty-seventh Conference on Neural Information Processing Systems*.
- Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. 2022b. [ReAct: Synergizing reasoning and acting in language models](#). volume abs/2210.03629.
- Yao Yao, Zuchao Li, and Hai Zhao. 2024. [GoT: Effective graph-of-thought reasoning in language models](#). In *Findings of the Association for Computational Linguistics: NAACL 2024*, pages 2901–2921, Mexico City, Mexico. Association for Computational Linguistics.
- Shaoqing Zhang, Zhuosheng Zhang, Kehai Chen, Xinbei Ma, Muyun Yang, Tiejun Zhao, and Min Zhang. 2024a. [Dynamic planning for llm-based graphical user interface automation](#). In *Findings of the Association for Computational Linguistics: EMNLP 2024*, pages 1304–1320.
- Zhehao Zhang, Jiaao Chen, and Diyi Yang. 2024b. [Darg: Dynamic evaluation of large language models via adaptive reasoning graph](#). *Preprint*, arXiv:2406.17271.
- Zhuosheng Zhang, Yao Yao, Aston Zhang, Xiangru Tang, Xinbei Ma, Zhiwei He, Yiming Wang, Mark Gerstein, Rui Wang, Gongshen Liu, and Hai Zhao. 2023. [Igniting language intelligence: The hitchhiker’s guide from chain-of-thought reasoning to language agents](#). *Preprint*, arXiv:2311.11797.
- Zirui Zhao, Wee Sun Lee, and David Hsu. 2024. [Large language models as commonsense knowledge for large-scale task planning](#). *Advances in Neural Information Processing Systems*, 36.
- Yaowei Zheng, Richong Zhang, Junhao Zhang, Yanhan Ye, Zheyang Luo, Zhangchi Feng, and Yongqiang Ma. 2024. [LlamaFactory: Unified efficient fine-tuning of 100+ language models](#). In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 3: System Demonstrations)*, Bangkok, Thailand. Association for Computational Linguistics.
- Andy Zhou, Kai Yan, Michal Shlapentokh-Rothman, Haohan Wang, and Yu-Xiong Wang. 2024. [Language agent tree search unifies reasoning acting and planning in language models](#).

A 提示模板

下面展示提示模板及示例。

图规划的提示模板

给定一组转换规则，每条规则包括源节点（材料或子任务）、目标节点（得到的材料或任务）、完成转换所需时间，以及对应成本。你的目标是从初始节点规划到目标节点的路径，支持并行转换，在尽可能短的时间内得到目标节点，同时最小化总成本。

输入格式：

- 转换规则：一个字典列表，每个字典表示一条规则，包含：

- source：源节点列表，即转换的前置条件。
- target：目标节点列表，即转换结果。
- time：完成转换所需时间，取整数。
- cost：转换所需成本，取整数。

- 初始节点：字符串列表，表示开始时可用的节点。

- 目标节点：字符串，表示需要获得的节点。

输出格式：

- Plan（计划）：子任务列表，每个子任务是一个 JSON 对象，包含以下字段：

- name：正在完成的子任务或节点名称。默认名称格式为“Subtask”后接序号。
- source：该子任务涉及的源节点列表。源必须是已经拥有、或能够通过先前步骤得到的产物。
- target：该子任务得到的目标节点。源和目标都必须符合给定规则，不得假设或自行创建。
- dependencies：执行该子任务前需要完成的依赖列表，即其他子任务名称。这保证子任务间的执行顺序，且依赖必须提供本子任务所需的源。

重要要求：

- 生成的 JSON 必须严格符合 JSON 格式，并严格遵守以下规则：

- 所有键和值都必须用双引号括起来。^a
- 数组中的所有元素必须用逗号分隔。
- JSON 的所有字段必须完整、格式正确，不能缺失元素或包含错误元素。

- 全部规划步骤必须遵循给定规则。

- 涉及的全部材料必须符合给定规则。

你的任务是按指定 JSON 格式生成最终计划，同时最小化完成时间和总成本。不要给出任何实现代码。

下面给出一个示例，帮助理解任务：

{graph_planning_example}

现在，根据以下转换规则、初始节点和目标节点，给出支持并行转换的最优计划，使目标节点能够在最短时间内、以最小总成本获得。只包含以最少成本最快完成所必需的步骤，不要加入额外或冗余转换步骤。

任务：

{task}

你的任务是按指定 JSON 格式生成最终计划。不要给出任何实现代码。

^a译注：原提示要求“所有键和值”用双引号，但其示例的 time/cost 为未加引号的数字；后续抽取提示也要求数值。示例代码保留原样，其中一处 cost 字段缺少开头双引号，另一段使用单引号 id 和尾随逗号，均属于原文的格式问题。

查询规划的提示模板

对于输入任务，给出能够获得目标的最优计划。在耗时最短的前提下，最小化成本。没有依赖的项目可以并行完成，以提高整体效率。

以 JSON 格式给出最终方案：

- Plan（计划）：子任务列表，每个子任务是一个 JSON 对象，包含以下字段：

- name：正在完成的子任务或节点名称。默认名称格式为“Subtask”后接序号。
- source：该子任务涉及的源节点列表。源必须是已经拥有、或能够通过先前步骤得到的产物。
- target：该子任务得到的目标节点。源和目标都必须符合给定规则，不得假设或自行创建。
- dependencies：执行该子任务前需要完成的依赖列表，即其他子任务名称。这保证子任务间的执行顺序，且依赖必须提供本子任务所需的源。下面给出一个示例：

输入：

{query_example}

输出：

{query_example_plan}

输入：

```
{task}  
输出:
```

从查询抽取图的提示模板

任务：从非结构化工作流叙述中抽取结构化转换规则。目标：识别文本中节点间的全部转换。对于每次转换，抽取：

- 源节点，即前置条件。
- 目标节点，即结果。
- 时间，即持续时长。
- 成本，以数值表示的资源单位。

此外，确定 initial_source（起始节点）和 target（最终节点）。

输入：描述工作流程的故事。示例措辞可能包括：

- “从 [NodeA] 到 [NodeB]，耗时 X 天，成本为 Y 单位”。
- “当 [NodeA] 和 [NodeB] 都就绪时，可以在 X 天内、以 Y 单位成本完成 [NodeC]”。
- 捷径，例如“直接从 [NodeA] 到 [NodeC]，耗时 X 天，成本为 Y 单位”。

输出：一个 JSON 对象，包含：

1. "rules"：转换规则列表，每条规则包含：
- "id"：从 0 开始的连续整数。

- "source"：节点 ID 列表，例如 ["N1"]。

- "target"：节点 ID 列表，例如 ["N2"]。

- "time"：数值。

- "cost"：数值。

2. "initial_source"：起始节点 ID 列表，例如 ["N1"]。

3. "target"：最终节点 ID，例如"N8"。

你的任务：将以下故事转换为上述 JSON 格式，并确保：

1. 获取全部转换，包括多源依赖与捷径。
2. 节点 ID（例如 N1、N2）严格保留原来的写法。
3. 时间和成本必须为数值。
4. 严格遵循 JSON 模式。

输入故事示例：

```
{query_example}
```

输出示例：

```
{query_example_plan}
```

输入故事：

```
{task}
```

输出：

从图生成查询的提示模板

将这个抽象任务转换为现实场景中的具体任务，注意以下要求：

1. 没有依赖的任务可以并行执行。
2. 用完整自然语言表述指令，不要显式列出规则。
3. 只要有一条路径能够达到最终目标，就视为成功。
4. 必须完全获得某条规则的源，才能执行该规则并得到其目标。
5. 严格遵守给定规则，确保故事中的规则与所提供规则一一对应，故事中的规则总数必须等于任务中的规则总数。
6. 必须在故事中明确说明每条规则的时间和成本。
7. 只需把规则写成故事，不要给出额外评估意见或介绍性说明。

以下是另一个任务的示例，供参考：

输入：

```
{query_example_plan}
```

输出：

```
{query_example}
```

输入：

```
{task}
```

输出：

示例

graph_planning_example:

任务:

```
{
  "rules": [
    {
      "source": ["N1"],
      "target": ["N2"],
      "time": 3,
      "cost": 1
    },
    {
      "source": ["N6"],
      "target": ["N3"],
      "time": 4,
      "cost": 1
    },
    {
      "source": ["N2", "N3"],
      "target": ["N4"],
      "time": 2,
      "cost": 1
    },
    {
      "source": ["N4"],
      "target": ["N5"],
      "time": 1,
      "cost": 1
    },
    {
      "source": ["N2"],
      "target": ["N5"],
      "time": 5,
      "cost": 1
    }
  ],
  "initial_source": ["N1", "N6"],
  "target": "N5"
}
```

预期输出:

```
[
  {
    "name": "Subtask1",
    "source": ["N1"],
    "target": ["N2"],
    "dependencies": []
  },
  {
    "name": "Subtask2",
    "source": ["N6"],
    "target": ["N3"],
    "dependencies": []
  },
  {
    "name": "Subtask3",
    "source": ["N2", "N3"],
    "target": ["N4"],
    "dependencies": ["Subtask1", "Subtask2"]
  },
  {
    "name": "Subtask4",
    "source": ["N4"],
    "target": ["N5"],
    "dependencies": ["Subtask3"]
  }
]
```

```
}
]
```

query_example:

在一个繁忙的城市建设项目中，需要协调多个场地，尽快且经济地建成“核心区（N9）”。项目从“基础设施区（N1）”“高架区（N3）”和“住宅区（N7）”三个场地开始，各有不同任务。“基础设施区（N1）”到“桥梁区（N2）”需要 3 天、成本为 1；“高架区（N3）”到“建筑区（N4）”需要 3 天、成本为 1。“桥梁区（N2）”到“道路区（N5）”需要 4 天、成本为 1，也可以直接到“设施区（N6）”，需要 8 天、成本为 1。“建筑区（N4）”与“道路区（N5）”共同建设“设施区（N6）”，需要 2 天、成本为 1。“住宅区（N7）”到“市中心区（N8）”需要 5 天、成本为 1；“建筑区（N4）”也可以直接到达市中心区，需要 1 天、成本为 1。当“设施区（N6）”和“市中心区（N8）”都就绪时，可以联合完成“核心区（N9）”，需要 2 天、成本为 1。“基础设施区（N1）”还有一条绕过其他区域、直接到“核心区（N9）”的捷径，需要 15 天、成本为 1。项目团队可以根据资源和进度选择最高效路线。

query_example_plan:

```
{"rules": [{ "id": 0, "source": ["N1"], "target": ["N2"], "time": 3, "cost": 1 }, { "id": 1,
  ↳ "source": ["N3"], "target": ["N4"], "time": 3, "cost": 1 }, { "id": 2, "source": ["N2"],
  ↳ "target": ["N5"], "time": 4, "cost": 1 }, { "id": 3, "source": ["N4", "N5"], "target":
  ↳ ["N6"], "time": 2, "cost": 1 }, { "id": 4, "source": ["N2"], "target": ["N6"], "time": 8,
  ↳ "cost": 1 }, { "id": 5, "source": ["N7"], "target": ["N8"], "time": 5, "cost": 1 }, {
  ↳ "id": 6, "source": ["N4"], "target": ["N8"], "time": 1, "cost": 1 }, { "id": 7, "source":
  ↳ ["N6", "N8"], "target": ["N9"], "time": 2, "cost": 1 }, { "id": 8, "source": ["N1"],
  ↳ "target": ["N9"], "time": 15, "cost": 1 }, ], "initial_source": ["N1", "N3", "N7"],
  ↳ "target": "N9"}
```

B 文本查询统计

图 4 展示合成查询数据的统计。

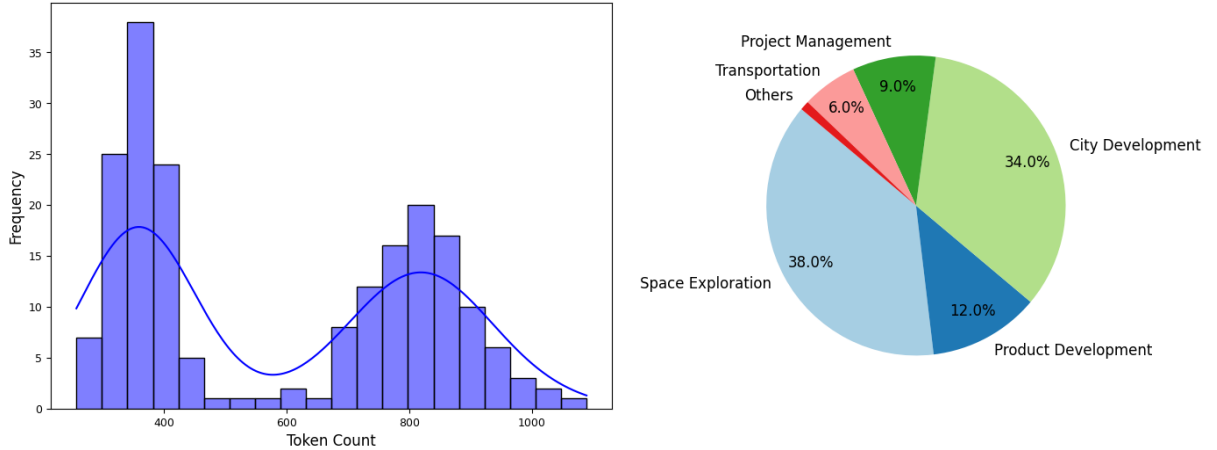


图 4: 合成查询统计。左侧柱状图为 Token 数量分布，右侧饼图为主题分布。图内文字译释：Frequency 为频数，Token Count 为 Token 数；主题为 Project Management（项目管理，9.0%）、Transportation（交通运输，6.0%）、Others（其他，1.0%）、City Development（城市建设，34.0%）、Product Development（产品开发，12.0%）、Space Exploration（太空探索，38.0%）。原图中其他主题的比例未直接标字，这里由总比例补足。

C 训练设置

训练使用 LLaMa Factory 框架 (Zheng et al., 2024)。

对于不混合可行解的 SFT 数据，给定图 G ，输入与输出为 $(x, y) = (G, p_{\text{opt}})$ ；使用混合数据时，对于 G ，输入与输出同时包括 $(x, y) = (G, p_{\text{opt}})$ 和 (G, p_{second}) 。DPO 数据的输入 x 、偏好输出 y_w 和不偏好输出 y_l 分别由 $(G, p_{\text{opt}}, p_{\text{second}})$ 给出。

SFT 的训练损失如下：

$$\mathcal{L}_{\text{SFT}}(\theta) = -\mathbb{E}_{(x, y) \sim \mathcal{D}_{\text{SFT}}} \sum_{t=1}^{|y|} \log P_{\theta}(y_t | x, y_{<t}). \quad (12)$$

DPO 的训练损失如下：

$$\mathcal{L}_{\text{DPO}} = -\mathbb{E}_{(x, y_w, y_l) \sim \mathcal{D}_{\text{DPO}}} \log \sigma \left(\beta \left(\log \frac{\pi_{\theta}(y_w|x)}{\pi_{\text{ref}}(y_w|x)} - \log \frac{\pi_{\theta}(y_l|x)}{\pi_{\text{ref}}(y_l|x)} \right) \right). \quad (13)$$

详细超参数设置列于下表。

名称	值
截断长度	8,192
训练轮数	10
每个设备的批大小	1
梯度累积步数	4
学习率	1e-6
学习率调度类型	cosine
预热比例	0.1
BF16	true

表 6: 详细训练超参数。

模型	成功率		最优率		时间比		成本比	
	<i>r</i>	<i>m</i>	<i>r</i>	<i>m</i>	<i>r</i>	<i>m</i>	<i>r</i>	<i>m</i>
Claude 3.5 Sonnet	-0.96	-0.99	-0.82	-0.82	0.92	1.02	0.94	0.99
GPT-4o	-0.99	-1.02	-0.94	-0.97	0.99	1.09	0.98	1.1
Llama-3.1-8B-Instruct	-0.95	-0.96	-0.95	-1.01	0.79	0.92	0.93	0.97
Llama-3.1-8B-Instruct-Trained	-0.94	-1.04	-0.89	-0.86	0.94	1.04	0.93	1.0
Qwen2.5-7B-Instruct	-0.96	-0.98	-0.96	-0.95	0.96	0.97	0.66	0.66
Qwen2.5-7B-Instruct-Trained	-0.99	-0.98	-0.80	-0.82	0.81	0.76	0.96	0.98

表 7: 各项指标与节点数之间的相关系数 (*r*) 和斜率 (*m*)。

D 边数变化下的状态分布

E 查询示例

附录 A 展示了一个现实场景中的查询及对应计划。

F 实验补充说明

表 7 和表 8 展示四项归一化指标随节点数及边数变化的相关系数和斜率的绝对值。³

使用最小-最大归一化，将指标缩放到 [0,1] 区间：

$$y_{\text{norm}} = \frac{y - y_{\min}}{y_{\max} - y_{\min}} \quad (14)$$

其中，*y* 表示节点数、边数、成功实例数、最优实例数、平均时间比和平均成本比的原始值。

节点数、边数 (*X*) 与指标 (*Y*) 之间的 Pearson 相关系数 (*r*) 计算为：

$$r_{XY} = \frac{\sum_{i=1}^n (x_i - \bar{x})(y_i - \bar{y})}{\sqrt{\sum_{i=1}^n (x_i - \bar{x})^2} \sqrt{\sum_{i=1}^n (y_i - \bar{y})^2}}. \quad (15)$$

³译注：本段与正文称为绝对值，但原表实际保留正负号；相关系数列的数值位于 [-1,1]，超过 1 的部分属于斜率列。这里保留原表与表述，不擅自改为绝对值。

模型	节点数	成功率		最优率		时间比		成本比	
		r	m	r	m	r	m	r	m
Claude 3.5 Sonnet	10	0.27	0.15	-0.44	-0.28	0.50	0.26	0.43	0.23
	30	-0.74	-0.56	-0.71	-0.55	0.81	0.59	0.78	0.59
Llama-3.1-8B-Instruct-Trained	10	0.05	0.03	-0.28	-0.17	0.58	0.34	0.45	0.24
	30	-0.39	-0.29	-0.62	-0.37	0.45	0.34	0.60	0.43

表 8: 不同节点规模下, 边数变化与各项指标之间的相关系数 (r) 和斜率 (m)。

通过线性回归计算最佳拟合直线的斜率 (m):

$$m = \frac{n \sum x_i y_i - \sum x_i \sum y_i}{n \sum x_i^2 - (\sum x_i)^2}. \quad (16)$$

最终报告的结果均四舍五入到两位小数。

对于由子计划 p 构成的给定计划 P , 其并行执行时间与顺序执行时间之比为:

$$\text{Ratio} = \frac{\Omega_{\text{time}}(P)}{\sum_{p \in P} \tau_p}. \quad (17)$$

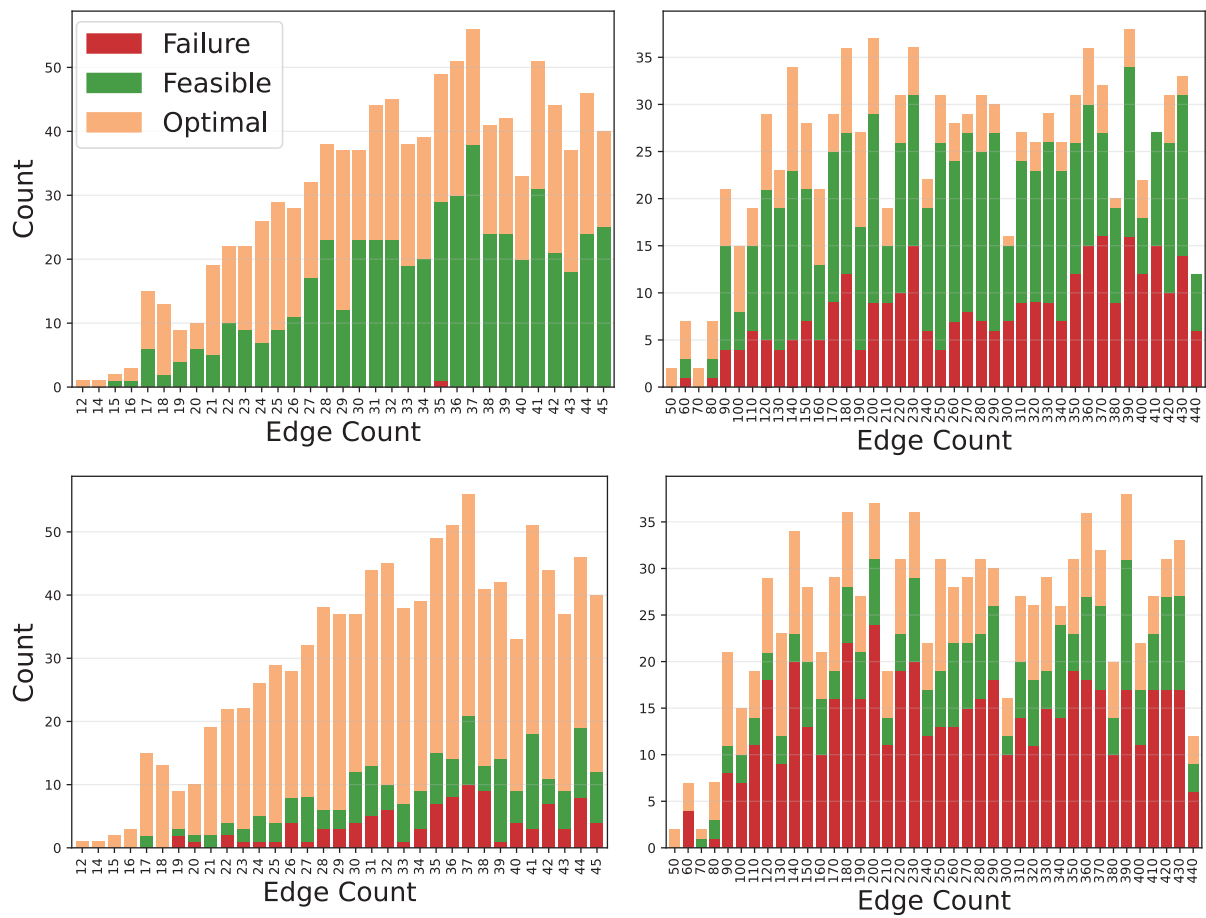


图 5: Claude 与已训练 Llama 在不同边数下的表现。纵轴为各状态（失败、可行、最优）对应的实例数，横轴为按一定间隔分组的边数。图内文字译释: *Count* 为实例数, *Edge Count* 为边数; *Failure*、*Feasible*、*Optimal* 分别为失败、可行但非最优、最优。